

High-Performance Reverse Proxy and Load Balancer

B.Tech Project Proposal | Indian Institute of Technology Patna, Department of CSE

Aditya Prakash | Roll No. 2302CS11 | Dual Degree (B.Tech + M.Tech), CSE | aditya.2302cs11@iitp.ac.in

Abstract. This project proposes the design and implementation of a **high-performance reverse proxy and load balancer**, inspired by systems such as NGINX and Envoy, built from first principles. It will support multiple load-balancing strategies, active/passive health checking, connection reuse, rate limiting, response caching, and gradual (canary) traffic shifting, exposed via a real-time monitoring dashboard. A companion load-testing tool will benchmark the system on latency, throughput, error rate, and resource usage.

1 Introduction & Motivation

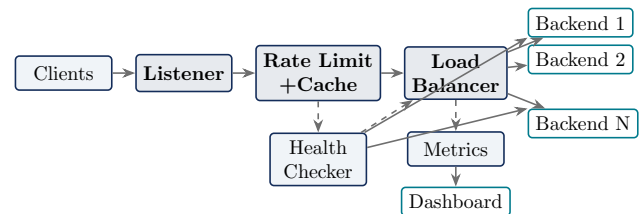
Reverse proxies and load balancers sit at the entry point of nearly every large-scale web system, deciding where a request goes, how fast it is served, and how failures are handled. This project reconstructs the core mechanisms of production proxies — connection handling, health checking, load distribution, and traffic shaping — as a focused, rigorously benchmarked implementation.

- **Systems depth** — Touches networking, concurrency, and I/O multiplexing end-to-end.
- **Real-world relevance** — Directly applicable to SRE / backend engineering practice.
- **Measurable outcomes** — Quality is quantifiable via throughput and tail latency.
- **Prior experience** — Builds on prior distributed-systems and observability work.

2 Objectives

- O1.** Reverse proxy core forwarding HTTP(S) to multiple backends with minimal added latency.
- O2.** Implement and compare load-balancing algorithms (round robin, weighted RR, least connections, IP hash).
- O3.** Active + passive health checking with automatic failover/recovery.
- O4.** Connection pooling / keep-alive reuse.
- O5.** Rate limiting (token bucket) and response caching.
- O6.** Weighted canary / blue-green traffic shifting.
- O7.** Real-time monitoring dashboard.
- O8.** Independent load-generation tool for rigorous benchmarking.

3 System Architecture



- **Listener** — Non-blocking, event-driven I/O sustaining thousands of concurrent connections.
- **LB Engine** — Pluggable strategy interface across four algorithms.
- **Health Checker** — Periodic active probes + passive failure signals drive automatic failover/recovery.
- **Conn. Pool** — Persistent backend connections avoid repeated TCP/TLS handshakes.
- **Metrics** — Prometheus-compatible export to a live dashboard.

4 Load Balancing Strategies

Strategy	Description
Round Robin	Even sequential distribution across healthy backends.
Weighted RR	Higher-weight backends receive proportionally more requests.
Least Conn.	Routes to backend with fewest active connections.
IP Hash	Client IP hashed to backend for sticky sessions.

5 Reliability & Traffic Control

- **Automatic Failover** — Unhealthy backends removed/reinstated within one health-check interval.
- **Rate Limiting** — Token-bucket limiter protects backends from overload.
- **Response Caching** — In-memory LRU cache

with TTL reduces backend load and latency.

► **Canary Routing** — Weighted traffic split (e.g. 95/5) for safe incremental rollout.

6 Technology Stack

Layer	Technology
Proxy Core	Go / C++20 (epoll event loop)
Load Tester	Go, concurrent request generator
Monitoring Dashboard	Prometheus + Grafana Next.js, TypeScript, WebSockets
Deployment	Docker, Docker Compose / Kubernetes

7 Testing & Evaluation

A Go-based load-testing tool will generate thousands of concurrent requests (constant, burst, and ramp-up patterns) against the proxy. Metrics captured: **response time** (p50/p95/p99), **requests/sec**, **error rate**, **CPU/memory usage**, and **failover time**. Each load-balancing algorithm will be benchmarked under identical backend and traffic conditions to compare fairness, tail latency, and resilience to uneven load, with results presented as comparative throughput-vs-concurrency and latency-percentile plots.

8 Timeline (16 Weeks)

Weeks	Phase
1–4	Core proxy: connection layer, basic forwarding
5–8	Load balancing algorithms, health checks & failover
9–12	Rate limiting, caching, canary routing, conn. pooling
13–16	Load-tester, dashboard, benchmarking & report

9 Expected Outcomes

1. Functional proxy with 4 LB strategies, health-based failover, connection pooling, caching, rate limiting, and canary routing.
2. Independent high-concurrency load-testing tool.
3. Real-time monitoring dashboard.
4. Quantitative comparison report of LB strategies and failure-handling behaviour.
5. Documented, extensible open-source codebase suitable for future work (HTTP/2, gRPC, service mesh).

10 References

- [1] NGINX Inc., *NGINX Architecture and Load Balancing*.
- [2] HAProxy Technologies, *Configuration Manual*.
- [3] Envoy Proxy Contributors, *Architecture Overview*.
- [4] Kleppmann, *Designing Data-Intensive Applications*, O'Reilly, 2017.
- [5] Prometheus Authors, *Documentation*, prometheus.io.